Bug Hunt 2: Binary Search Tree

What the program does

`bst.erl` implements a **binary search tree** of integer keys. The empty tree is the atom `empty`. A non-empty node is `{node, Key, Left, Right}` where every key in `Left` is less than `Key` and every key in `Right` is greater than `Key`.

The module exports:

- `new()` — return an empty tree.
- `insert(Tree, K)` — return a new tree with `K` inserted. Duplicates are ignored (the tree is unchanged).
- `insert_list(Tree, Keys)` — insert every key in `Keys` left-to-right.
- `contains(Tree, K)` — `true` if `K` is in the tree, `false` otherwise.
- `size(Tree)` — number of nodes.
- `min(Tree)` / `max(Tree)` — smallest / largest key. Undefined on `empty`.
- `to_list(Tree)` — in-order traversal. Should produce keys in sorted (ascending) order.
- `height(Tree)` — height of the tree. Empty has height `-1`, a single node has height `0`.

`main.erl` builds a BST from the list `[5, 3, 8, 1, 9, 4, 7, 2]` and exercises every operation.

The catch

This program contains **exactly 3 bugs** that prevent it from producing the expected output. The bugs are all *semantic* — the program compiles without errors and runs to completion without crashing.

You may not run the program until you have submitted your answer. Trace by reading.

Expected output

```
inserted: [5,3,8,1,9,4,7,2]
size: 8
min: 1
max: 9
height: 3
contains 4: true
contains 6: false
contains 9: true
contains 5: true
in-order: [1,2,3,4,5,7,8,9]
```

Your write-up

For each bug, record:

1. **Location** — file name and line number(s).
2. **Diagnosis** — in 1–2 sentences, what is wrong.
3. **Fix** — the corrected line(s).
4. **Symptom** — which line(s) of the expected output differ from what the buggy program actually prints, and why this specific bug causes that specific symptom.